Laporan Tugas Besar

IF2224 Teori Bahasa Formal dan Otomata

Milestone 2 - Syntax Analysis

Semester II tahun 2025/2026

Oleh:

Kelompok ReguExceptional:

Wafiq Hibban Robbany	13524016
Muhammad Nafis Habibi	13524018
An-Dafa Anza Avansyah	13524038
Wildan Abdurrahman Ghazali	13524054

Laboratorium Ilmu dan Rekayasa Komputasi

Program Studi Teknik Informatika

Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung

7 Mei 2026

Daftar Isi

Daftar Isi **2**
Dasar Teori **3**
A. Syntax Analysis / Parser 3
B. Algoritma Recursive Descent 3
C. Parse Tree 4
Perancangan dan Implementasi **5**
A. Aturan Grammar Bahasa Arion 5
B. Implementasi 10
Parser 10
Pengujian **16**
Kesimpulan dan Saran **30**
A. Kesimpulan 30
B. Saran 30
Lampiran **32**
Referensi **33**

Dasar Teori

A. Syntax Analysis / Parser

1. Definisi Syntax Analysis / Parser

   Syntax analysis atau parser merupakan tahap kedua dalam proses kompilasi setelah lexical analysis. Pada tahap ini, token-token yang dihasilkan pada tahap lexical analysis diperiksa apakah susunannya sesuai dengan aturan tata bahasa atau *grammar* bahasa pemrograman yang digunakan. Parser bertujuan memastikan bahwa urutan token membentuk struktur sintaks yang valid, mendeteksi dan melaporkan kesalahan sintaks, serta mempersiapkan program untuk tahap kompilasi berikutnya, yaitu membuat sebuah parse tree

2. Keterkaitan dengan lexer

   Lexer yang bertugas menghasilkan token-token dari source code memiliki keterkaitan penting dengan parser. Token-token yang dihasilkan lexer akan diperiksa oleh parser dan dianalisis apakah urutan token sesuai dengan *grammar* bahasa pemrograman. Jika token-token tersebut membentuk susunan yang valid, parser akan menghasilkan parse tree sebagai keluaran. Dengan demikian, parser menjadi penghubung antara representasi token hasil lexical analysis dan representasi struktur program yang lebih bermakna secara sintaksis.

B. Algoritma *Recursive Descent*

Algoritma *recursive descent* adalah algoritma *parsing top-down* yang bekerja dengan menurunkan grammar dari simbol awal menuju token-token input. Disebut *recursive descent* karena proses parsing dilakukan secara rekursif mengikuti struktur *grammar*. Setiap non-terminal pada grammar dapat dipandang sebagai bagian yang akan dikenali oleh parser, kemudian bagian tersebut dapat memanggil pengenalan non-terminal lain sesuai aturan produksinya.

Dalam pendekatan ini, *parser* membaca token dari awal ke akhir dan mencoba mencocokkannya dengan aturan *grammar*. Jika token yang sedang dibaca sesuai dengan

aturan yang diharapkan, proses *parsing* dilanjutkan ke token berikutnya. Jika token tidak sesuai, parser dapat menyatakan bahwa terjadi kesalahan sintaks.

C. Parse Tree

Parse tree adalah representasi lengkap dari struktur sintaks program berdasarkan grammar yang digunakan. Setiap node dalam parse tree merepresentasikan bagian dari aturan grammar, baik berupa simbol terminal maupun non-terminal. Karena itu, parse tree menunjukkan secara eksplisit bagaimana token-token hasil lexical analysis membentuk struktur program yang sesuai dengan aturan grammar.

Parse tree digunakan untuk menggambarkan hirarki program Arion mulai dari node akar seperti <program>, kemudian turun ke bagian-bagian seperti <program-header>, <declaration-part>, <compound-statement>, <statement-list>, hingga ekspresi dan token-token penyusunnya. Struktur ini penting karena dapat memperlihatkan hubungan antarbagian program secara jelas.

Perancangan dan Implementasi

A. Aturan Grammar Bahasa Arion

Berikut adalah tabel aturan *grammar* bahasa Arion:

Nama Node	Deskripsi	Aturan Produksi
`<program>`	Node root yang merepresentasikan keseluruhan program Arion	`<program> → <program-header> + <declaration-part> + <compound-statement> + period`
`<program-header>`	Bagian kepala program yang berisi nama program	`<program-header> → programsy + ident + semicolon`
`<declaration-part>`	Bagian deklarasi yang dapat berisi const, type, var, procedure, dan function	`<declaration-part> → (<const-declaration>)* + (<type-declaration>)* + (<var-declaration>)* + (<subprogram-declaration>)*`
`<const-declaration>`	Deklarasi konstanta	`<const-declaration> → constsy + (ident + eql + <constant> + semicolon)+`
`<constant>`	Konstanta dapat berupa identifier, integer, real, character, atau string	`<constant> → charcon | string | ((plus | minus)? + (ident | intcon | realcon))`
`<type-declaration>`	Deklarasi tipe data baru	`<type-declaration> → typesy + (ident + eql + <type> + semicolon)+`
`<var-declaration>`	Deklarasi variabel	`<var-declaration> → varsy + (<identifier-list> + colon + <type> + semicolon)+`
`<identifier`	Daftar identifier	`<identifier-list> → ident +`

-list>	yang dipisahkan koma	(comma + ident)*
<type>	Tipe data yang terdefinisi atau didefinisikan pengguna	<type> → ident | <array-type> | <range> | <enumerated> | <record-type>
<array-type>	Definisi tipe array	<array-type> → arraysy + lbrack + (<range> | ident) + rbrack + ofsy + <type>
<range>	Rentang nilai untuk array atau subrange	<range> → <constant> + period + period + <constant>
<enumerated>	Nilai tertentu yang berada di dalam list	<enumerated> → lparent + ident + (comma + ident)* + rparent
<record-type>	Tipe data template yang menyerupai class dalam OOP	<record-type> → recordsy + <field-list> + endsy
<field-list>	Penjabaran atribut atau field dari record	<field-list> → <field-part> + (semicolon + <field-part>)*
<field-part>	Satu bagian atribut atau field dari record	<field-part> → <identifier-list> + colon + <type>
<subprogram-declaration>	Deklarasi prosedur atau fungsi	<subprogram-declaration> → <procedure-declaration> | <function-declaration>
<procedure-declaration>	Deklarasi prosedur	<procedure-declaration> → proceduresy + ident + (<formal-parameter-list>)? + semicolon + <block> + semicolon
<function-declaration>	Deklarasi fungsi	<function-declaration> → functionsy + ident +

		(<formal-parameter-list>)? + colon + ident + semicolon + <block> + semicolon
<block>	Bagian kode yang mengandung deklarasi dan statement	<block> → <declaration-part> + <compound-statement>
<formal-parameter-list>	Daftar parameter formal	<formal-parameter-list> → lparent + <parameter-group> + (semicolon + <parameter-group>)* + rparent
<parameter-group>	Daftar parameter yang disertai tipe data parameter	<parameter-group> → <identifier-list> + colon + (ident | <array-type>)
<compound-statement>	Blok statement yang diawali begin dan diakhiri end	<compound-statement> → beginsy + <statement-list> + endsy
<statement-list>	Daftar statement yang dipisahkan semicolon	<statement-list> → <statement> + (semicolon + <statement>)*
<statement>	Deskripsi aksi yang harus dilakukan program komputer	<statement> → (<assignment-statement> | <if-statement> | <case-statement> | <while-statement> | <repeat-statement> | <for-statement> | <procedure/function-call>)?
<variable>	Variabel yang dapat berbentuk identifier, elemen array, atau field record	<variable> → ident + (<component-variable>)*
<component-	Komponen variabel dalam bentuk elemen	<component-variable> →

variable>	array atau field record	(lbrack + <index-list> + rbrack) | (period + ident)
<index-list>	Penunjuk indeks dalam array	<index-list> → (intcon | charcon | ident) + (comma + <index-list>)*
<assignment-statement>	Statement penugasan nilai ke variabel	<assignment-statement> → <variable> + becomes + <expression>
<if-statement>	Statement kondisional	<if-statement> → ifsy + <expression> + thensy + <statement> + (elsy + <statement>)?
<case-statement>	Statement kondisional dengan banyak kemungkinan kondisi	<case-statement> → casesy + <expression> + ofsy + <case-block> + endsy
<case-block>	Deklarasi case dan statement yang harus dilakukan	<case-block> → <constant> + (comma + <constant>)* + colon + <statement> + (semicolon + <case-block>?)*
<while-statement>	Perulangan dengan kondisi di awal	<while-statement> → whilesy + <expression> + dosy + <statement>
<repeat-statement>	Perulangan dengan kondisi di akhir	<repeat-statement> → repeatsy + <statement-list> + untilsy + <expression>
<for-statement>	Perulangan dengan counter	<for-statement> → forsy + ident + becomes + <expression> + (tosy | downtosy) + <expression> + dosy + <statement>

`<procedure/function-call>`	Pemanggilan prosedur atau fungsi	`<procedure/function-call> → ident + (lparent + <parameter-list>? + rparent)`
`<parameter-list>`	Daftar parameter aktual saat pemanggilan	`<parameter-list> → <expression> + (comma + <expression>)*`
`<expression>`	Ekspresi yang menghasilkan nilai	`<expression> → <simple-expression> + (<relational-operator> + <simple-expression>)?`
`<simple-expression>`	Ekspresi tanpa operator relasional	`<simple-expression> → (plus | minus)? + <term> + (<additive-operator> + <term>)*`
`<term>`	Bagian ekspresi dengan prioritas lebih tinggi	`<term> → <factor> + (<multiplicative-operator> + <factor>)*`
`<factor>`	Unit terkecil dalam ekspresi	`<factor> → ident | intcon | realcon | charcon | string | (lparent + <expression> + rparent) | (notsy + <factor>) | <procedure/function-call> | <variable>`
`<relational-operator>`	Operator perbandingan	`<relational-operator> → eql | neq | gtr | geq | lss | leq`
`<additive-operator>`	Operator penjumlahan, pengurangan, atau logika OR	`<additive-operator> → plus | minus | orsy`
`<multiplicative-operator>`	Operator perkalian, pembagian, modulo, atau logika AND	`<multiplicative-operator> → times | rdiv | idiv | imod | andsy`

B. Implementasi

Implementasi dilakukan dalam bahasa C++ dan dibuat dalam *namespace* "arion" agar terpisah dari *standard library* C++.

Parser

Parser diimplementasikan dalam bentuk kelas bersama dengan beberapa kelas pembantu seperti Symbol dan ParseNode. Kelas Symbol digunakan untuk merepresentasikan entitas terminal maupun non-terminal, dengan menyimpan atribut seperti *id*, *label*, dan nilai. Sedangkan kelas ParseNode merepresentasikan sebuah simpul pada *Parse Tree* yang menyimpan sebuah Symbol dan *vector* simpul anak-anaknya. ParseNode juga memiliki metode toString yang mengimplementasikan penelusuran *depth-first* untuk mencetak pohon sintaksis secara hierarkis dan terstruktur ke bentuk string.

Kelas utama Parser menerima masukan berupa struktur data *vector* yang berisi sekumpulan objek Token hasil keluaran dari tahapan analisis leksikal. Parser membaca *list of tokens* ini secara sekuensial menggunakan penunjuk indeks current_. Sebelum memulai *parsing* atau saat memajukan indeks, parser akan melewati token-token yang diabaikan dalam sintaksis menggunakan metode skipIgnoredTokens().

Algoritma yang digunakan adalah *Recursive Descent*, di mana seluruh *non-terminal* dari *grammar* bahasa Arion didefinisikan dalam sebuah enum NonTerminal dan masing-masing direpresentasikan oleh sebuah fungsi khusus. Setiap fungsi ini bertugas membangun ParseNode bersangkutan, lalu mencocokkan token yang sesuai atau memanggil fungsi *parsing* lainnya untuk membangun struktur cabang dari pohon.

Proses recursive descent pada parser dimulai dari fungsi parse() yang memanggil parseProgram() sebagai simbol awal grammar. Fungsi parseProgram() kemudian menurunkan aturan grammar ke fungsi-fungsi lain seperti parseProgramHeader(), parseDeclarationPart(), dan parseCompoundStatement(). Setiap fungsi parsing membentuk sebuah ParseNode, lalu menambahkan child node berupa terminal maupun

non-terminal sesuai aturan produksi grammar. Jika aturan produksi membutuhkan terminal tertentu, parser menggunakan expect() untuk mencocokkan token saat ini. Jika aturan produksi membutuhkan non-terminal lain, parser akan memanggil fungsi parsing lain secara rekursif.

Selama parsing, parser menggunakan indeks current_ untuk menunjuk token yang sedang diproses. Fungsi peek(), peekNext(), dan lookahead() digunakan untuk melihat token tanpa langsung mengonsumsinya, sedangkan advance() digunakan untuk mengonsumsi token dan berpindah ke token berikutnya. Mekanisme lookahead digunakan untuk menentukan cabang grammar yang tepat, terutama ketika token saat ini berupa ident, karena ident dapat menjadi awal dari assignment statement, variable, atau procedure/function call. Apabila token yang ditemukan tidak sesuai dengan grammar, parser akan memanggil syntaxError() dan melempar ParserError agar kesalahan sintaks dapat ditampilkan secara jelas.

Pada parsing ekspresi, recursive descent dibagi ke beberapa tingkat fungsi, yaitu parseExpression(), parseSimpleExpression(), parseTerm(), dan parseFactor(). Pembagian ini membuat parser dapat mempertahankan prioritas operator, dengan operator relasional berada pada tingkat paling luar, operator penjumlahan dan pengurangan pada tingkat simple-expression, serta operator perkalian, pembagian, modulo, dan and pada tingkat term. Dengan demikian, proses recursive descent tidak hanya memvalidasi urutan token, tetapi juga membangun struktur parse tree yang sesuai dengan hierarki grammar bahasa Arion.

Untuk proses validasi token, Parser memiliki fungsi-fungsi seperti peek(), peekNext(), dan lookahead() untuk melihat token selanjutnya tanpa mengonsumsinya. Fungsi advance() digunakan untuk mengonsumsi token dan memajukan penunjuk. Terdapat juga fungsi expect(tokenType) yang secara ketat akan mengonsumsi token dengan memvalidasi apakah tipe token saat ini sesuai dengan aturan *grammar* yang diharapkan.

Apabila token yang diterima tidak sesuai dengan aturan *grammar*, Parser akan memanggil fungsi syntaxError(). Fungsi ini akan melempar sebuah *exception* khusus bernama ParserError beserta informasi token yang bermasalah dan token apa yang sebenarnya diharapkan, sehingga proses kompilasi dapat dihentikan dengan pesan *error handling* yang informatif.

Berikut definisi dari kelas-kelas utama Parser pada src/arion/Parser.hpp:

```
class Symbol {
    private:
        int id_;
        std::string label_;
        std::string value_;
        bool isTerminal_;

    public:
        Symbol(int id, std::string label, bool isTerminal =
false, std::string value = "");
        std::string toString() const;
        bool isTerminal() const;
    };

    class ParseNode {
    public:
        ParseNode() = default;
        explicit ParseNode(Symbol symbol);

        ParseNode &addChild(ParseNode child);
        Symbol getSymbol() const;
        std::string getLabel() const;
        const std::vector<ParseNode> &getChildren() const;
        std::string toString() const;

    private:
        Symbol symbol_;
        std::vector<ParseNode> children_;
        std::string toStringHelper(int depth,
std::vector<bool> &isLast) const;
    };

    class Parser {
    public:
        Parser() = default;
        explicit Parser(std::vector<Token> tokens);
```

```
        void setTokens(std::vector<Token> tokens);
        ParseNode parse();

    private:
        enum NonTerminal {
            PROGRAM = 2000,
            PROGRAM_HEADER,
            DECLARATION_PART,
            CONST_DECLARATION,
            CONSTANT,
            TYPE_DECLARATION,
            VAR_DECLARATION,
            IDENTIFIER_LIST,
            TYPE,
            ARRAY_TYPE,
            RANGE,
            ENUMERATED,
            RECORD_TYPE,
            FIELD_LIST,
            FIELD_PART,
            SUBPROGRAM_DECLARATION,
            PROCEDURE_DECLARATION,
            FUNCTION_DECLARATION,
            BLOCK,
            FORMAL_PARAMETER_LIST,
            PARAMETER_GROUP,
            COMPOUND_STATEMENT,
            STATEMENT_LIST,
            STATEMENT,
            ASSIGNMENT_STATEMENT,
            IF_STATEMENT,
            CASE_STATEMENT,
            CASE_BLOCK,
            WHILE_STATEMENT,
            REPEAT_STATEMENT,
            FOR_STATEMENT,
            PROCEDURE_FUNCTION_CALL,
            PARAMETER_LIST,
            EXPRESSION,
            SIMPLE_EXPRESSION,
            TERM,
            FACTOR,
            VARIABLE,
            COMPONENT_VARIABLE,
            INDEX_LIST,
            RELATIONAL_OPERATOR,
            ADDITIVE_OPERATOR,
            MULTIPLICATIVE_OPERATOR
```

```
        };

        const Token &peek() const;
        const Token &peekNext() const;
        const Token &lookahead(std::size_t offset) const;
        bool isAtEnd() const;
        bool check(int tokenType) const;
        bool checkNext(int tokenType) const;
        Token advance();
        Token expect(int tokenType);

        ParseNode variableNode(NonTerminal variable) const;
        ParseNode terminalNode(const Token &token) const;
        std::string variableName(NonTerminal variable) const;
        std::string terminalName(int tokenType) const;
        std::string tokenName(const Token &token) const;
        void syntaxError(const std::string &expectedName)
const;
        void skipIgnoredTokens();

        ParseNode parseProgram();
        ParseNode parseProgramHeader();
        ParseNode parseDeclarationPart();
        ParseNode parseConstDeclaration();
        ParseNode parseConstant();
        ParseNode parseTypeDeclaration();
        ParseNode parseVarDeclaration();
        ParseNode parseIdentifierList();
        ParseNode parseType();
        ParseNode parseArrayType();
        ParseNode parseRange();
        ParseNode parseEnumerated();
        ParseNode parseRecordType();
        ParseNode parseFieldList();
        ParseNode parseFieldPart();
        ParseNode parseSubprogramDeclaration();
        ParseNode parseProcedureDeclaration();
        ParseNode parseFunctionDeclaration();
        ParseNode parseBlock();
        ParseNode parseFormalParameterList();
        ParseNode parseParameterGroup();
        ParseNode parseCompoundStatement();
        ParseNode parseStatementList();
        ParseNode parseStatement();
        ParseNode parseAssignmentStatement();
        ParseNode parseIfStatement();
        ParseNode parseCaseStatement();
        ParseNode parseCaseBlock();
```

```
        ParseNode parseWhileStatement();
        ParseNode parseRepeatStatement();
        ParseNode parseForStatement();
        ParseNode parseProcedureOrFunctionCall();
        ParseNode parseParameterList();
        ParseNode parseExpression();
        ParseNode parseSimpleExpression();
        ParseNode parseTerm();
        ParseNode parseFactor();
        ParseNode parseVariable();
        ParseNode parseComponentVariable();
        ParseNode parseIndexList();
        ParseNode parseRelationalOperator();
        ParseNode parseAdditiveOperator();
        ParseNode parseMultiplicativeOperator();

        std::vector<Token> tokens_;
        std::size_t current_ = 0;
        Token eofToken_{Tokenizer::TOKEN_EOF, ""};
    };

    class ParserError : public std::runtime_error {
    public:
        explicit ParserError(const std::string &message);
    };
```

Pengujian

Kasus uji dapat dilihat di bagan berikut dan folder test/milestone-2/ di repositori GitHub

Tabel Hasil Pengujian	
1.	Input
| | ```
program Hello;

var
  a, b: integer;

begin
  a := { Hello Saya Nafis }  5;
  b := a + 10;
  writeln('Result = ', b);
end.
```  |
| | Output |
```

```
<program>
├── <program-header>
│   ├── programsy
│   ├── ident (Hello)
│   └── semicolon
├── <declaration-part>
│   └── <var-declaration>
│       ├── varsy
│       ├── <identifier-list>
│       │   ├── ident (a)
│       │   ├── comma
│       │   └── ident (b)
│       ├── colon
│       ├── <type>
│       │   └── ident (integer)
│       └── semicolon
├── <compound-statement>
│   ├── beginsy
│   ├── <statement-list>
│   │   ├── <statement>
│   │   │   └── <assignment-statement>
│   │   │       ├── <variable>
│   │   │       │   └── ident (a)
│   │   │       ├── becomes
│   │   │       └── <expression>
│   │   │           └── <simple-expression>
│   │   │               └── <term>
│   │   │                   └── <factor>
│   │   │                       └── intcon (5)
│   │   ├── semicolon
│   │   ├── <statement>
│   │   │   └── <assignment-statement>
│   │   │       ├── <variable>
│   │   │       │   └── ident (b)
│   │   │       ├── becomes
│   │   │       └── <expression>
│   │   │           └── <simple-expression>
│   │   │               ├── <term>
│   │   │               │   └── <factor>
│   │   │               │       └── ident (a)
│   │   │               ├── <additive-operator>
│   │   │               │   └── plus
│   │   │               └── <term>
│   │   │                   └── <factor>
│   │   │                       └── intcon (10)
│   │   ├── semicolon
│   │   ├── <statement>
│   │   │   └── <procedure/function-call>
│   │   │       ├── ident (writeln)
│   │   │       ├── lparent
│   │   │       ├── <parameter-list>
│   │   │       │   ├── <expression>
│   │   │       │   │   └── <simple-expression>
```

<table>
<tr><td></td><td>

```
54  │   │   │       │   │       └── <term>
55  │   │   │       │   │           └── <factor>
56  │   │   │       │   │               └── string ('Result = ')
57  │   │   │       │   ├── comma
58  │   │   │       │   └── <expression>
59  │   │   │       │       └── <simple-expression>
60  │   │   │       │           └── <term>
61  │   │   │       │               └── <factor>
62  │   │   │       │                   └── ident (b)
63  │   │   │       └── rparent
64  │   │   └── semicolon
65  │   └── endsy
66  └── period
```

</td></tr>
<tr><td colspan="2">Hasil Uji Test Case 1: Sesuai</td></tr>
<tr><td>2.</td><td>Input</td></tr>
</table>

```
program Types;

const
  Min == 1;
  Max == 3;

type
  SmallRange == Min .. Max;
  Pair == record
    x: integer
  end;
  Numbers == array[SmallRange] of integer;

var
  p: Pair;
  n: Numbers;

begin
end.
```

Output

```
<program>
├── <program-header>
│   ├── programsy
│   ├── ident (Types)
│   └── semicolon
├── <declaration-part>
│   ├── <const-declaration>
│   │   ├── constsy
│   │   ├── ident (Min)
│   │   ├── eql
│   │   ├── <constant>
│   │   │   └── intcon (1)
│   │   ├── semicolon
│   │   ├── ident (Max)
│   │   ├── eql
│   │   ├── <constant>
│   │   │   └── intcon (3)
│   │   └── semicolon
│   ├── <type-declaration>
│   │   ├── typesy
│   │   ├── ident (SmallRange)
│   │   ├── eql
│   │   ├── <type>
│   │   │   └── <range>
│   │   │       ├── <constant>
│   │   │       │   └── ident (Min)
│   │   │       ├── period
│   │   │       ├── period
│   │   │       └── <constant>
│   │   │           └── ident (Max)
│   │   ├── semicolon
│   │   ├── ident (Pair)
│   │   ├── eql
│   │   ├── <type>
│   │   │   └── <record-type>
│   │   │       ├── recordsy
│   │   │       ├── <field-list>
│   │   │       │   └── <field-part>
│   │   │       │       ├── <identifier-list>
│   │   │       │       │   └── ident (x)
│   │   │       │       ├── colon
│   │   │       │       └── <type>
│   │   │       │           └── ident (integer)
│   │   │       └── endsy
│   │   ├── semicolon
│   │   ├── ident (Numbers)
│   │   ├── eql
│   │   ├── <type>
```

```
│   │   │   └── <array-type>
│   │   │       ├── arraysy
│   │   │       ├── lbrack
│   │   │       ├── ident (SmallRange)
│   │   │       ├── rbrack
│   │   │       ├── ofsy
│   │   │       └── <type>
│   │   │           └── ident (integer)
│   │   └── semicolon
│   └── <var-declaration>
│       ├── varsy
│       ├── <identifier-list>
│       │   └── ident (p)
│       ├── colon
│       ├── <type>
│       │   └── ident (Pair)
│       ├── semicolon
│       ├── <identifier-list>
│       │   └── ident (n)
│       ├── colon
│       ├── <type>
│       │   └── ident (Numbers)
│       └── semicolon
├── <compound-statement>
│   ├── beginsy
│   ├── <statement-list>
│   │   └── <statement>
│   └── endsy
└── period
```

Hasil Uji Test Case 2:	Sesuai
3.	Input

```
program Flow;

var
  i: integer;

begin
  i := 1;
  if i == 1 then
    i := i + 1
  else
    i := i - 1;
  while i < 3 do
    i := i + 1;
  repeat
    i := i - 1
  until i == 0;
  for i := 1 to 2 do
    i := i + 1
end.
```

Output

```
<program>
├── <program-header>
│   ├── programsy
│   ├── ident (Flow)
│   └── semicolon
├── <declaration-part>
│   └── <var-declaration>
│       ├── varsy
│       ├── <identifier-list>
│       │   └── ident (i)
│       ├── colon
│       ├── <type>
│       │   └── ident (integer)
│       └── semicolon
├── <compound-statement>
│   ├── beginsy
│   ├── <statement-list>
│   │   ├── <statement>
│   │   │   └── <assignment-statement>
│   │   │       ├── <variable>
│   │   │       │   └── ident (i)
│   │   │       ├── becomes
│   │   │       └── <expression>
│   │   │           └── <simple-expression>
│   │   │               └── <term>
│   │   │                   └── <factor>
│   │   │                       └── intcon (1)
│   │   ├── semicolon
│   │   ├── <statement>
│   │   │   └── <if-statement>
│   │   │       ├── ifsy
│   │   │       ├── <expression>
│   │   │       │   ├── <simple-expression>
│   │   │       │   │   └── <term>
│   │   │       │   │       └── <factor>
│   │   │       │   │           └── ident (i)
│   │   │       │   ├── <relational-operator>
│   │   │       │   │   └── eql
│   │   │       │   └── <simple-expression>
│   │   │       │       └── <term>
│   │   │       │           └── <factor>
│   │   │       │               └── intcon (1)
│   │   │       ├── thensy
│   │   │       ├── <statement>
│   │   │       │   └── <assignment-statement>
│   │   │       │       ├── <variable>
│   │   │       │       │   └── ident (i)
│   │   │       │       ├── becomes
│   │   │       │       └── <expression>
│   │   │       │           └── <simple-expression>
│   │   │       │               ├── <term>
│   │   │       │               │   └── <factor>
│   │   │       │               │       └── ident (i)
│   │   │       │               ├── <additive-operator>
│   │   │       │               │   └── plus
│   │   │       │               └── <term>
│   │   │       │                   └── <factor>
│   │   │       │                       └── intcon (1)
```

```
59  │   │   │       ├── elsesy
60  │   │   │       └── <statement>
61  │   │   │           └── <assignment-statement>
62  │   │   │               ├── <variable>
63  │   │   │               │   └── ident (i)
64  │   │   │               ├── becomes
65  │   │   │               └── <expression>
66  │   │   │                   └── <simple-expression>
67  │   │   │                       ├── <term>
68  │   │   │                       │   └── <factor>
69  │   │   │                       │       └── ident (i)
70  │   │   │                       ├── <additive-operator>
71  │   │   │                       │   └── minus
72  │   │   │                       └── <term>
73  │   │   │                           └── <factor>
74  │   │   │                               └── intcon (1)
75  │   │   ├── semicolon
76  │   │   ├── <statement>
77  │   │   │   └── <while-statement>
78  │   │   │       ├── whilesy
79  │   │   │       ├── <expression>
80  │   │   │       │   ├── <simple-expression>
81  │   │   │       │   │   └── <term>
82  │   │   │       │   │       └── <factor>
83  │   │   │       │   │           └── ident (i)
84  │   │   │       │   ├── <relational-operator>
85  │   │   │       │   │   └── lss
86  │   │   │       │   └── <simple-expression>
87  │   │   │       │       └── <term>
88  │   │   │       │           └── <factor>
89  │   │   │       │               └── intcon (3)
90  │   │   │       ├── dosy
91  │   │   │       └── <statement>
92  │   │   │           └── <assignment-statement>
93  │   │   │               ├── <variable>
94  │   │   │               │   └── ident (i)
95  │   │   │               ├── becomes
96  │   │   │               └── <expression>
97  │   │   │                   └── <simple-expression>
98  │   │   │                       ├── <term>
99  │   │   │                       │   └── <factor>
100 │   │   │                       │       └── ident (i)
101 │   │   │                       ├── <additive-operator>
102 │   │   │                       │   └── plus
103 │   │   │                       └── <term>
```

```
│   │   │                       └── <factor>
│   │   │                           └── intcon (1)
│   │   ├── semicolon
│   │   ├── <statement>
│   │   │   └── <repeat-statement>
│   │   │       ├── repeatsy
│   │   │       ├── <statement-list>
│   │   │       │   └── <statement>
│   │   │       │       └── <assignment-statement>
│   │   │       │           ├── <variable>
│   │   │       │           │   └── ident (i)
│   │   │       │           ├── becomes
│   │   │       │           └── <expression>
│   │   │       │               └── <simple-expression>
│   │   │       │                   ├── <term>
│   │   │       │                   │   └── <factor>
│   │   │       │                   │       └── ident (i)
│   │   │       │                   ├── <additive-operator>
│   │   │       │                   │   └── minus
│   │   │       │                   └── <term>
│   │   │       │                       └── <factor>
│   │   │       │                           └── intcon (1)
│   │   │       ├── untilsy
│   │   │       └── <expression>
│   │   │           ├── <simple-expression>
│   │   │           │   └── <term>
│   │   │           │       └── <factor>
│   │   │           │           └── ident (i)
│   │   │           ├── <relational-operator>
│   │   │           │   └── eql
│   │   │           └── <simple-expression>
│   │   │               └── <term>
│   │   │                   └── <factor>
│   │   │                       └── intcon (0)
│   │   ├── semicolon
│   │   └── <statement>
│   │       └── <for-statement>
│   │           ├── forsy
│   │           ├── ident (i)
│   │           ├── becomes
│   │           ├── <expression>
│   │           │   └── <simple-expression>
│   │           │       └── <term>
│   │           │           └── <factor>
│   │           │               └── intcon (1)
│   │           ├── tosy
│   │           ├── <expression>
│   │           │   └── <simple-expression>
│   │           │       └── <term>
│   │           │           └── <factor>
│   │           │               └── intcon (2)
│   │           ├── dosy
│   │           └── <statement>
```

```
│   │   │   │
```

```
<program>
├── <program-header>
│   ├── programsy
│   ├── ident (Expr)
│   └── semicolon
├── <declaration-part>
│   └── <var-declaration>
│       ├── varsy
│       ├── <identifier-list>
│       │   ├── ident (a)
│       │   ├── comma
│       │   ├── ident (b)
│       │   ├── comma
│       │   └── ident (c)
│       ├── colon
│       ├── <type>
│       │   └── ident (integer)
│       └── semicolon
├── <compound-statement>
│   ├── beginsy
│   ├── <statement-list>
│   │   ├── <statement>
│   │   │   └── <assignment-statement>
│   │   │       ├── <variable>
│   │   │       │   └── ident (a)
│   │   │       ├── becomes
│   │   │       └── <expression>
│   │   │           └── <simple-expression>
│   │   │               └── <term>
│   │   │                   ├── <factor>
│   │   │                   │   ├── lparent
│   │   │                   │   ├── <expression>
│   │   │                   │   │   └── <simple-expression>
│   │   │                   │   │       ├── <term>
│   │   │                   │   │       │   └── <factor>
│   │   │                   │   │       │       └── ident (b)
│   │   │                   │   │       ├── <additive-operator>
│   │   │                   │   │       │   └── plus
│   │   │                   │   │       └── <term>
│   │   │                   │   │           └── <factor>
│   │   │                   │   │               └── ident (c)
│   │   │                   │   └── rparent
│   │   │                   ├── <multiplicative-operator>
│   │   │                   │   └── times
│   │   │                   └── <factor>
│   │   │                       └── intcon (2)
│   │   ├── semicolon
│   │   ├── <statement>
│   │   │   └── <assignment-statement>
│   │   │       ├── <variable>
│   │   │       │   └── ident (b)
│   │   │       ├── becomes
│   │   │       └── <expression>
│   │   │           └── <simple-expression>
│   │   │               └── <term>
│   │   │                   └── <factor>
│   │   │                       ├── notsy
│   │   │                       └── <factor>
```

```
59  │   │   │                       ├── lparent
60  │   │   │                       ├── <expression>
61  │   │   │                       │   ├── <simple-expression>
62  │   │   │                       │   │   └── <term>
63  │   │   │                       │   │       └── <factor>
64  │   │   │                       │   │           └── ident (a)
65  │   │   │                       │   ├── <relational-operator>
66  │   │   │                       │   │   └── lss
67  │   │   │                       │   └── <simple-expression>
68  │   │   │                       │       └── <term>
69  │   │   │                       │           └── <factor>
70  │   │   │                       │               └── ident (c)
71  │   │   │                       └── rparent
72  │   │   ├── semicolon
73  │   │   └── <statement>
74  │   │       └── <assignment-statement>
75  │   │           ├── <variable>
76  │   │           │   └── ident (c)
77  │   │           ├── becomes
78  │   │           └── <expression>
79  │   │               └── <simple-expression>
80  │   │                   └── <term>
81  │   │                       ├── <factor>
82  │   │                       │   └── ident (a)
83  │   │                       ├── <multiplicative-operator>
84  │   │                       │   └── idiv
85  │   │                       ├── <factor>
86  │   │                       │   └── intcon (2)
87  │   │                       ├── <multiplicative-operator>
88  │   │                       │   └── imod
89  │   │                       └── <factor>
90  │   │                           └── intcon (3)
91  │   └── endsy
92  └── period
```

Hasil Uji Test Case 4:	Sesuai

5.	Input

```
program KurangTitikKomaWoy;

var
  a: integer
  b: integer;

begin
  a := 1;
  b := 2;
end.
```

Output

```
Syntax error: unexpected token ident (b), expected semicolon
```

Hasil Uji Test Case 5:	Sesuai

Kesimpulan dan Saran

A. Kesimpulan

Berdasarkan implementasi dan pengujian yang telah dilakukan pada Milestone 2 tugas besar Arion Compiler, dapat ditarik beberapa kesimpulan sebagai berikut:

- Tahap *Syntax Analysis* untuk bahasa pemrograman Arion telah berhasil diimplementasikan menggunakan algoritma *Recursive Descent* secara modular menggunakan bahasa C++.
- Program *parser* mampu menerima masukan berupa daftar token yang dihasilkan dari tahapan *lexical analysis*, lalu memvalidasi urutan token tersebut berdasarkan aturan grammar yang telah di-*hardcode* ke dalam program.
- Dari hasil pemrosesan token, program berhasil mengonstruksi struktur hierarkis program dalam bentuk *Parse Tree* yang lengkap, serta mencetak hasilnya dengan format yang rapi dan terstruktur ke terminal maupun berkas teks (.txt).
- Program memiliki *error handling* yang baik. Apabila ditemukan ketidaksesuaian tata bahasa pada kode sumber, *parser* akan melempar *exception* berupa ParserError yang menampilkan pesan spesifik mengenai token apa yang bermasalah dan token apa yang sebenarnya diharapkan.

B. Saran

Berdasarkan hasil implementasi dan pengujian yang telah dilakukan, terdapat beberapa saran yang dapat dipertimbangkan untuk pengembangan tahap berikutnya:

- Parser yang telah dibuat sebaiknya terus dikembangkan agar dapat menangani lebih banyak variasi kasus uji, khususnya untuk struktur program Arion yang lebih kompleks seperti deklarasi bertingkat, pemanggilan fungsi/prosedur dengan banyak parameter, penggunaan *record*, *array*, serta kombinasi berbagai jenis statement dalam satu blok program.
- Mekanisme *error handling* dapat dikembangkan lebih lanjut agar tidak hanya menghentikan proses parsing pada kesalahan pertama, tetapi juga mampu

memberikan informasi lokasi kesalahan yang lebih detail, seperti nomor baris dan kolom token yang bermasalah.

- Struktur *Parse Tree* yang telah dihasilkan dapat dijadikan dasar untuk pengembangan program pada Milestone berikutnya, yaitu *semantic analysis*. Oleh karena itu, struktur *node* dan hubungan antar *node* sebaiknya tetap dijaga agar modular, konsisten, dan mudah diproses kembali pada tahap analisis semantik.

Lampiran

Link repository Github: https://github.com/NafisKreatif/RGX-Tubes-IF2224-2026

Tabel Pembagian Tugas

No.	Nama	NIM	Pembagian Tugas	Kontribusi (%)
1	Wafiq Hibban Robbany	13524016	Header dan struktur ParseNode, Implementasi fungsi *recursive descent*	27
2	Muhammad Nafis Habibi	13524018	Pengecekan dan revisi struktur ParseNode, integrasi ke program main	25
3	An-Dafa Anza Avansyah	13524038	Implementasi fungsi *recursive descent*	26
4	Wildan Abdurrahman Ghazali	13524054	Implementasi fungsi *recursive descent*	22

Referensi

[1] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Boston, MA: Addison-Wesley, 2006.

[2] J. E. Hopcroft, R. Motwani, and J. D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, 3rd ed. Boston, MA: Pearson Addison-Wesley, 2006.